CIS 1 – Introduction to Computers

hello, world!

Agenda

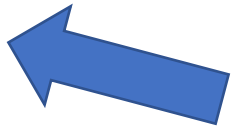
- About this class
- Textbook (sort of)
- Karel the Robot!

ABOUT THIS CLASS

- **Welcome to CIS 1 – Introduction to Computers!**
- If you have extensive previous programming experience, this is probably *NOT* the right course for you.
- We will be, however, doing *extensive* programming in this class – right from the start
- We will also learn the principles of computing, learning how computers work from the ground up
- We'll be using the Python programming language for most of the class, but the goal here isn't to learn Python -- it's to learn how to think like a computer scientist

ABOUT THIS CLASS

- **Welcome to CIS 1 – Introduction to Computers!**
- We'll be using the Python programming language for most of the class, but the goal here isn't to learn Python -- it's to learn how to think like a computer scientist



getting good at programming isn't about learning programming languages. It's about learning to think like a computer scientist

ABOUT THIS CLASS

practical details

- Meets Mondays and Wednesdays from 11:00 AM to 12:55 PM

- Although officially there's separate lecture and lab sessions, we'll be mixing lecture and lab together

ABOUT THIS CLASS

textbook

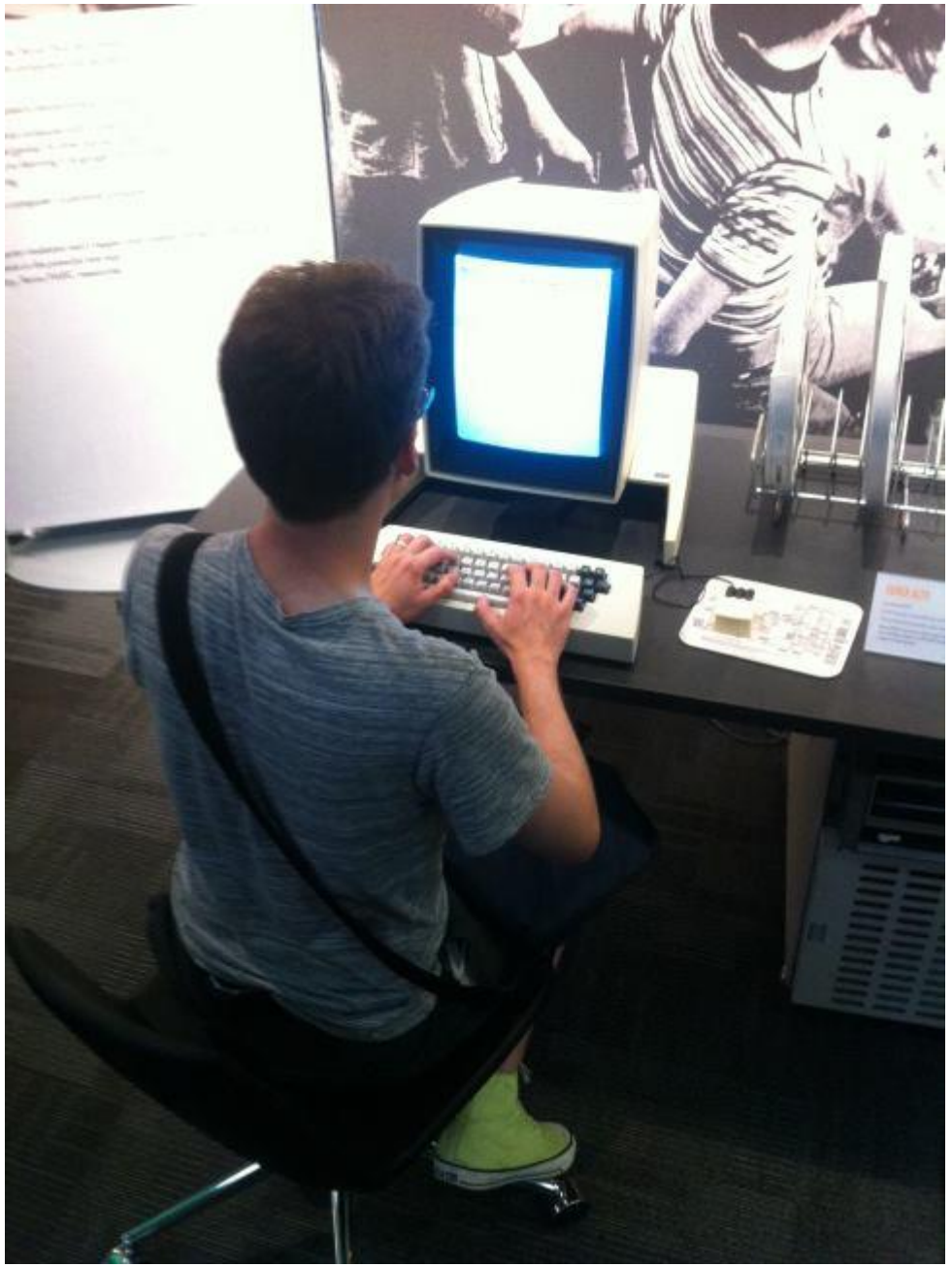
- We have NO PAID TEXTBOOK in this class
- Instead, we will be using the free textbook *How to Think Like a Computer Scientist*
- It's older (2012), but everything in it is relevant to this class



- Available at https://allendowney.github.io/ThinkPython/

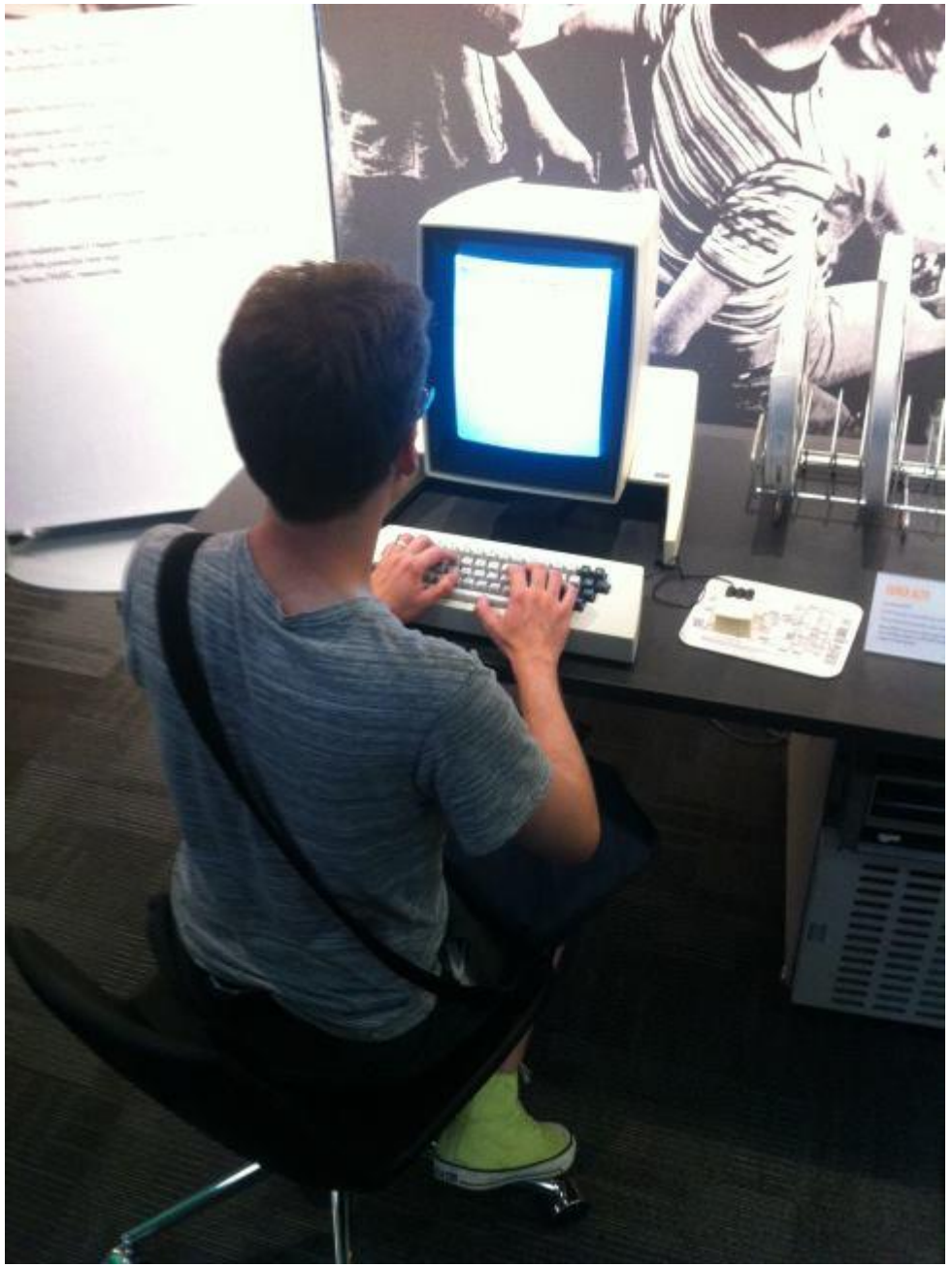
ABOUT ME

- Name: Ben Allen
- Preferred pronouns: he/him



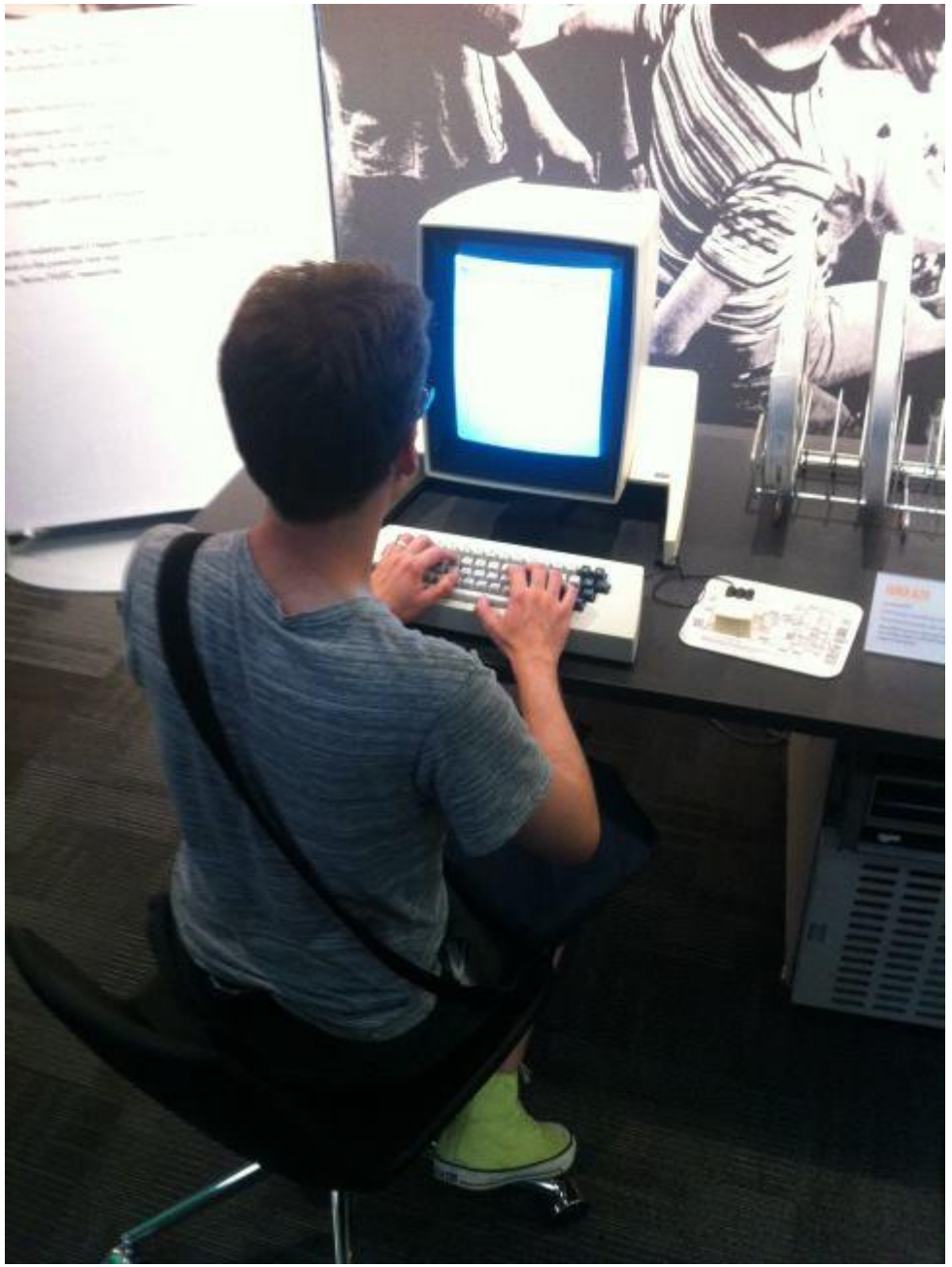
ABOUT ME

- Associate's degree– Bellevue Community College
- A few years back I got an M.S. in Computer Science from Stanford, focusing on CS education



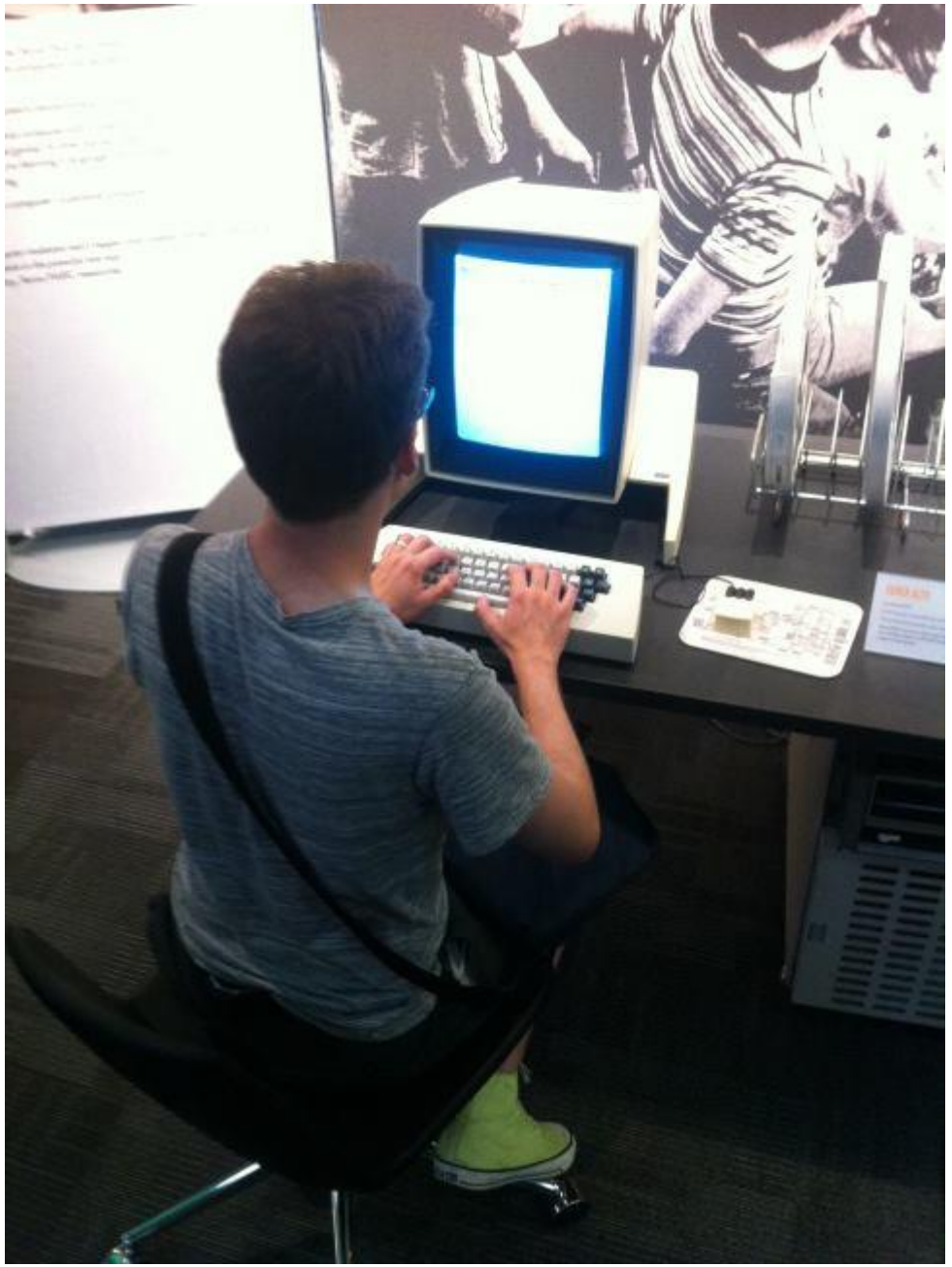
ABOUT ME

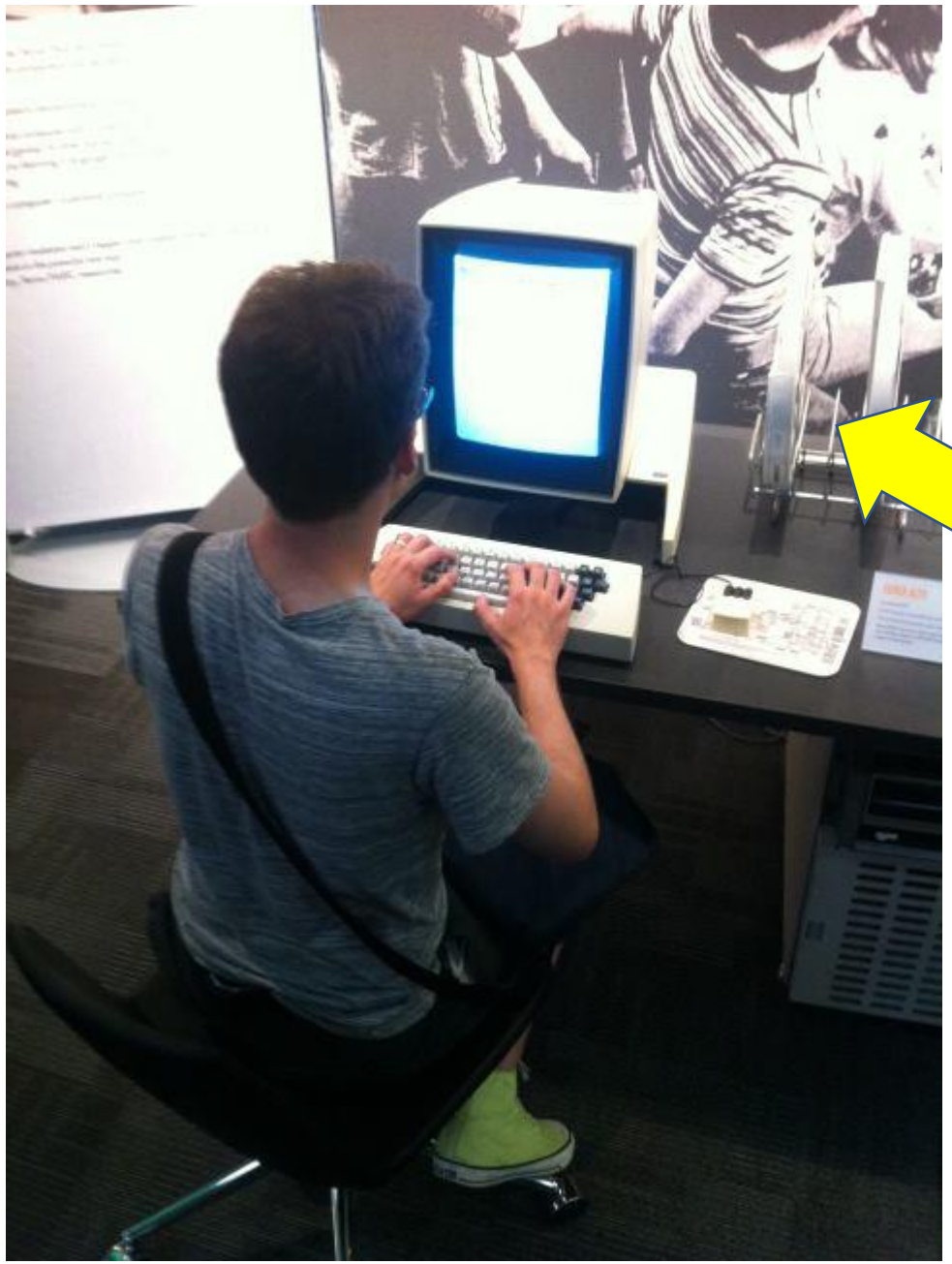
- Associate's degree– Bellevue Cmty Col.
- Before that, I got a PhD in the history of computing
- (which is why I was so excited about using this computer)



ABOUT ME

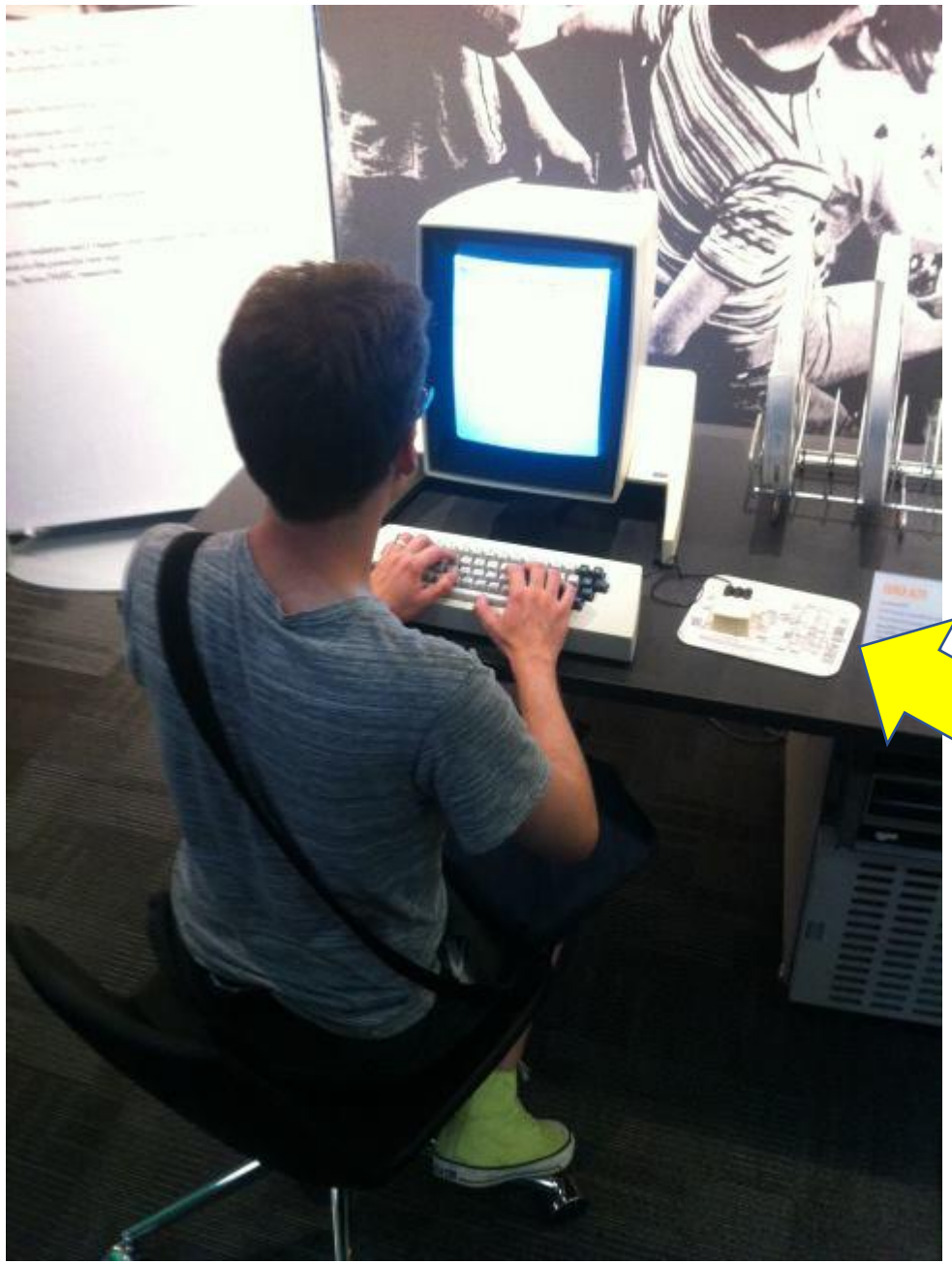
- Associate's degree– Bellevue Cmty Col.
- Before that, I got a PhD in the history of computing
- We will include *extensive* material from the history of computing in this class
- The best way to learn the concepts behind computing is to see how they developed





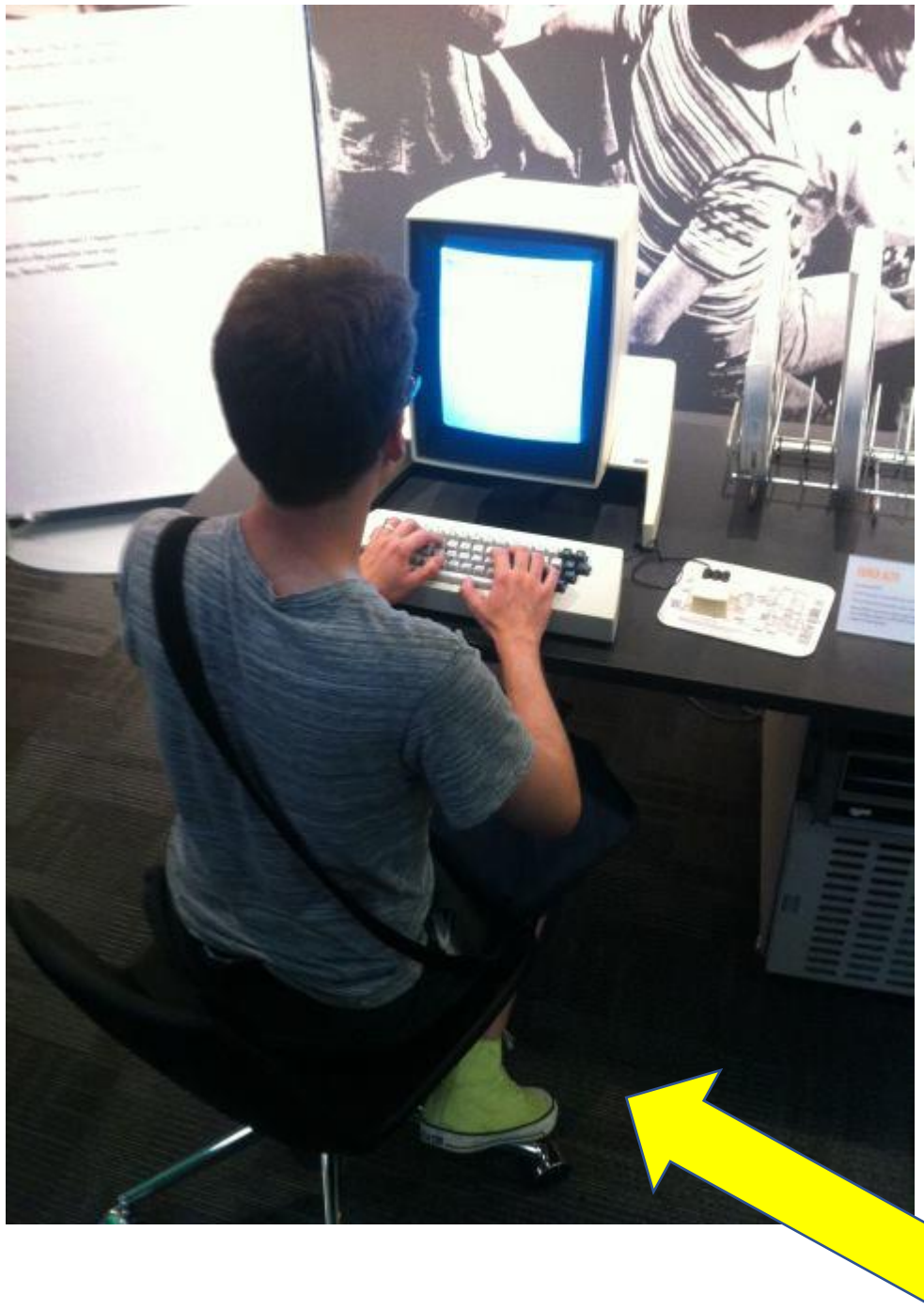
ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- First computer with a mouse!
- Built in 1973!



ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- First computer with a mouse!
- Built in 1973!



ABOUT THIS PICTURE

- The machine I'm sitting at here is a Xerox Alto
- It was the first computer with a graphical user interface (windows, icons, pointer)
- First computer with a mouse!
- Built in 1973!

also notice: my shoes are awesome

ACCESSIBILITY SERVICES CENTER (ASC)

Solano's office for disability services & accommodations

- If you have a disability, ASC can arrange accommodations for this class (extended test time, alternate media, assistive technology, etc.)
- Where: Building 400, Room 407
- Phone: (707) 864-7136 · Email: asc@solano.edu
- solano.edu/centers/accessibility-services-center
- Getting started: fill out the ASC Application for Services, send in documentation, then make an appointment with an ASC counselor
- Please contact them early — and feel free to talk to me, too



KAREL THE ROBOT

has escaped from Stanford

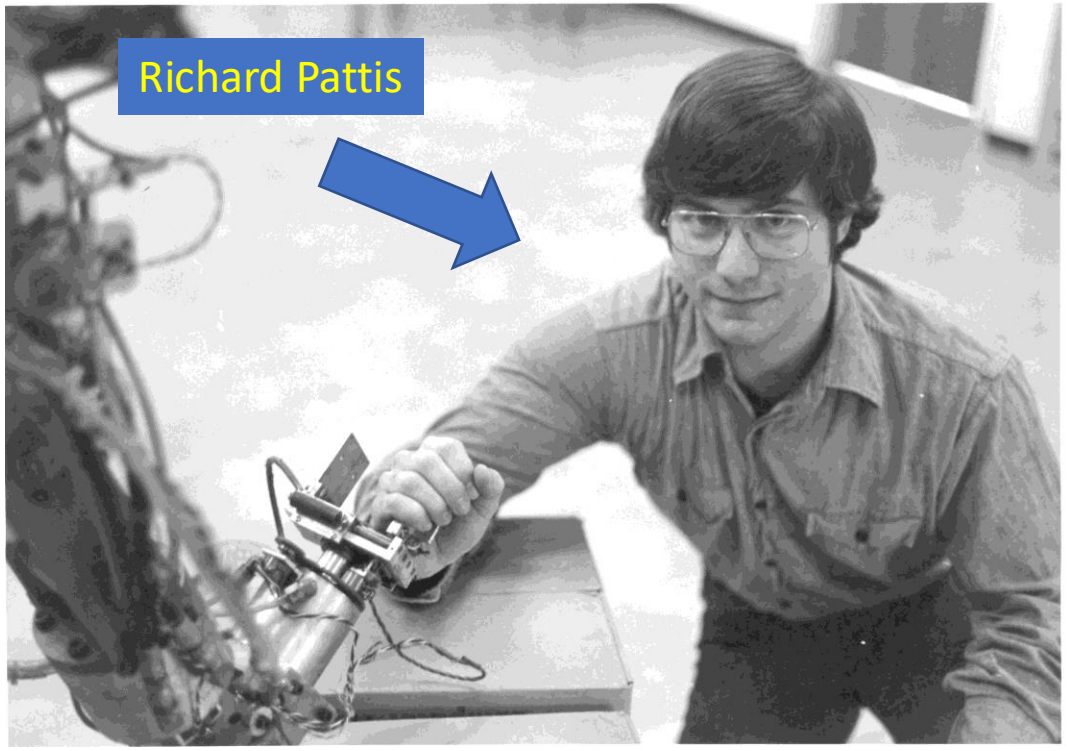
KAREL HISTORY

- Originally developed by a computer scientist named Richard Pattis while he was working on his master's degree at Stanford

- First released in 1981

- Karel has been used at Stanford ever since

- (I think) this is the first class at BCC to ever use Karel



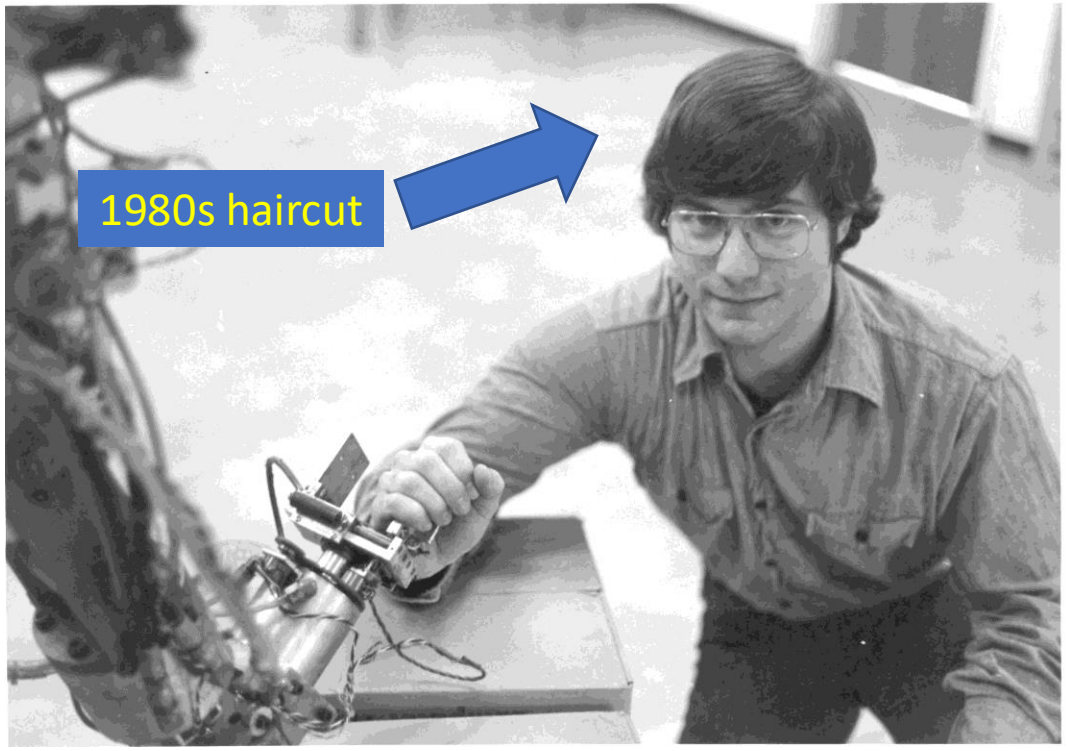
KAREL HISTORY

- Originally developed by a computer scientist named Richard Pattis while he was working on his master's degree at Stanford

- First released in 1981

- Karel has been used at Stanford ever since

- (I think) this is the first class at BCC to ever use Karel



ABOUT KAREL

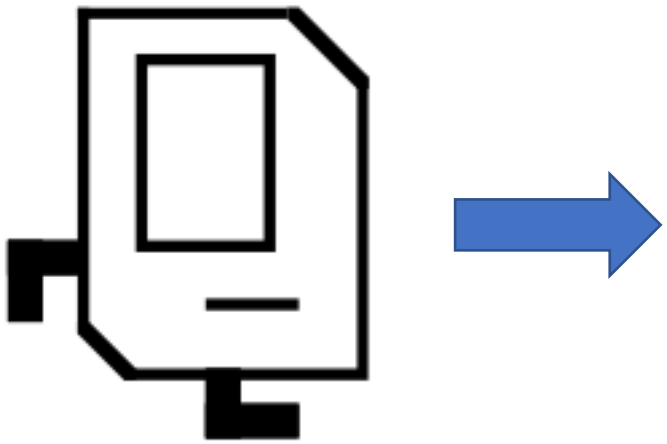


- Karel was born in 1981
- Karel looks a little bit like a 1980s Macintosh computer

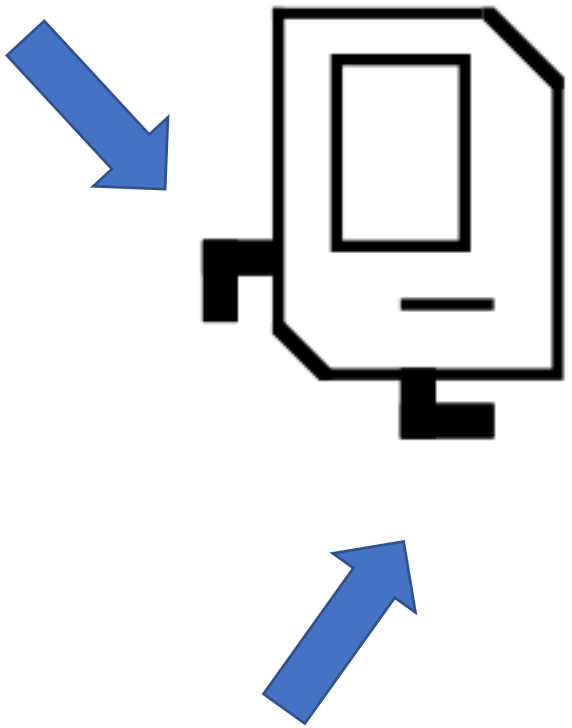


ABOUT KAREL

- Karel is facing RIGHT in this picture



ABOUT KAREL



- Karel is facing RIGHT in this picture
- These are Karel's feet

ABOUT KAREL



- Karel is named after Karel Čapek, the Czech playwright who invented the word "robot" back in 1920.

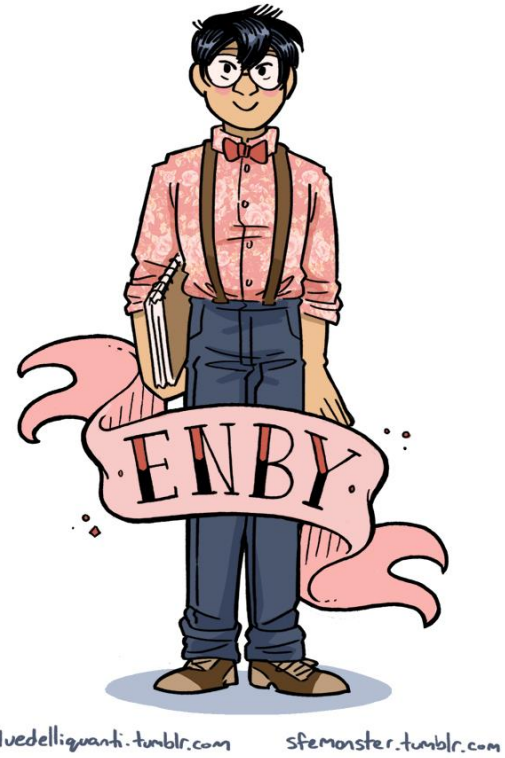
Rossum's Universal Robots. It's a pretty good play!



ABOUT KAREL



- Karel is a non-binary robot
- Karel uses she/her or they/their pronouns (either way is fine)



WHERE TO FIND KAREL



- We'll be using an online Karel IDE available here:
- https://ben-allen.github.io/karel-the-robot/karel.html
- THIS MAY BE BUGGY. I put it together pretty quickly and it hasn't been stress-tested by any classes yet

KAREL COMMANDS

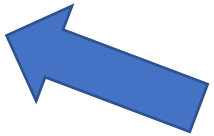
- Here are the first few commands you need to help Karel navigate their world
  - move()
  - turn_left()
  - put_beeper()
  - pick_beeper()



These are all FUNCTION CALLS. how do you know? the parentheses after each of them.
We'll talk about what a function call is later on

- If a command is by itself on a line, that's called a STATEMENT

```
move()
turn_left()
put_beeper()
```



each of these are statements

- When you click the run program button, Karel starts running statements starting at the line reading

```
def main():
```

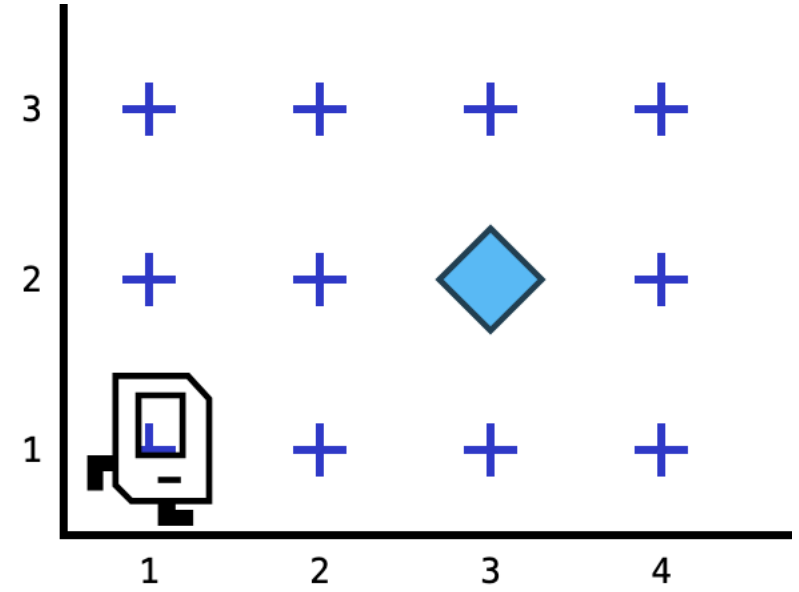
- "`def`" means "define", because it *defines* a function

- Text after `def main():` is *indented*. Indentation *matters* in Karel. Everything at one or more levels of indentation is considered *inside the main function*

exercise 1

- Write a program to make Karel:
  1. go to the square two squares ahead of their initial position, and one square up from their initial position
  2. drop a beeper there
  3. return to their initial position
  4. turn to face the way they were facing at the start

your goal



exercise 1

- When finished, put your code in a text file. Put *this exact line*:

```
# exercise 1
```

before the main function

- Karel also accepts a few COMPOUND STATEMENTS

```
while front_is_clear():
```



this is a CONDITIONAL STATEMENT

this compound statement executes *while* this conditional statement is TRUE

everything from while front_is_clear() to the next *non-indented line* is part of the while loop

notice
that front_is_clear() w is a
FUNCTION – it's got parentheses after it

```
while front_is_clear():
        move()
put_beeper()
```

move() is *inside* the while loop. You can tell, because it's indented

When you enter the loop, Karel checks the condition. If that condition is true, Karel runs all the statements inside the loop, then checks the condition again.

If the condition is still true, Karel runs the loop again! If it's false, Karel goes on to whatever's after the loop

put_beeper is *outside* the while loop. you know this because it is *not* indented

multiple levels of indentation

```
def main():
    # your code here...
    while front_is_clear():
        move()
    put_beeper()
```

notice that there are two levels of indentation here:

Everything in the *first* level of indentation is inside the
main() function

Everything inside the *second* level of indentation is inside *both* the main() function *and also* the while loop

what does this program do?

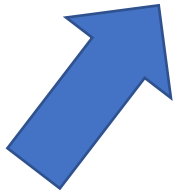
```
from karel.stanfordkarel import *
```

```
def main():
while front_is_clear():
        move()
        put_beeper()
```



new program

```
def main():
    while front_is_clear():
        move()
    put_beeper()
```

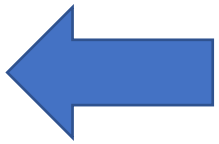


original program

exercise 2

```
from karel.stanfordkarel import *


def main():
while front_is_clear():
        move()
        put_beeper()
```

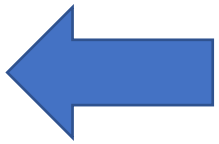


update this program to
put a beeper on *every* square
on the bottom row

exercise 2

```
from karel.stanfordkarel import *


def main():
while front_is_clear():
        move()
        put_beeper()
```



when done, put this in the same
text file as exercise 1, with *this exact line:*

```
# exercise 2
```

above the main function

syntax highlighting



```
def main():
    # your code here...
    while front_is_clear():
        move()
    put_beeper()
```



notice that some words are *highlighted* in the Karel interface

this indicates that they are *language keywords*

syntax highlighting helps you keep track of what parts of the program are which, and let you see the program's structure at a glance

all programmers' editors use syntax highlighting extensively

comments

```
def main():
    # your code here...
    while front_is_clear():
        move()
    put_beeper()
```



Text after # is considered part of a *comment*, and is ignored by Karel

Comments are ***very*** useful for helping programmers remember what complex pieces of code do

crashing

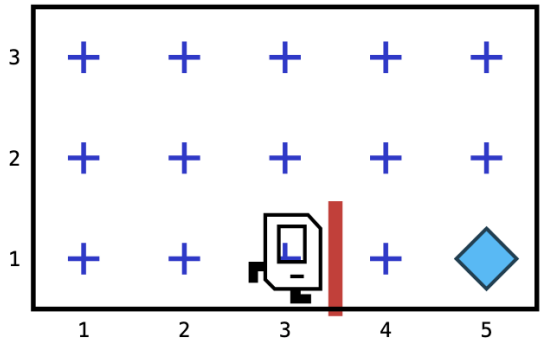
- Today we'll discuss two ways of making Karel crash – if Karel crashes, that means your program is buggy. Completed programs *must not crash*

crashing – walking into a wall

```
from karel.stanfordkarel import *


def main():
    # your code here...
    while front_is_clear():
        move()
    move()
```

Around the wall



▶ Run Program | Step | Pause | Reset | Speed | medium

Show Text Descriptions

```
Line 8: Karel tried to move east from (3, 1) but a wall is in the way.
```

crashing – walking into a wall

Line 8: Karel tried to move east from (3, 1) but a wall is in the way

Whoops! Karel walked into a wall. If Karel tries to move into a wall, program execution stops, and cannot be resumed

Even if Karel has completed their task, crashing is considered wrong.

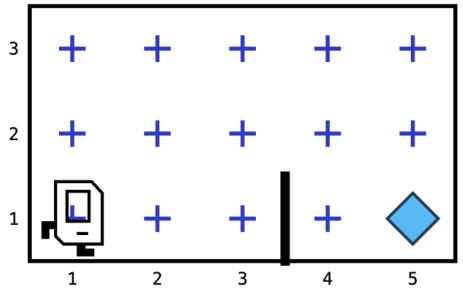
The interface will tell you on what line the crash occurred

crashing – picking up a beeper when there's no beeper present

```
from karel.stanfordkarel import *


def main():
    pick_beeper()
```

Around the wall



▶ Run Program | Step | Pause | Reset | Speed | medium

Show Text Descriptions

Line 5: Karel tried to pick up a beeper at (1, 1) but there isn't one there.

crashing – picking up a beeper when there's no beeper present

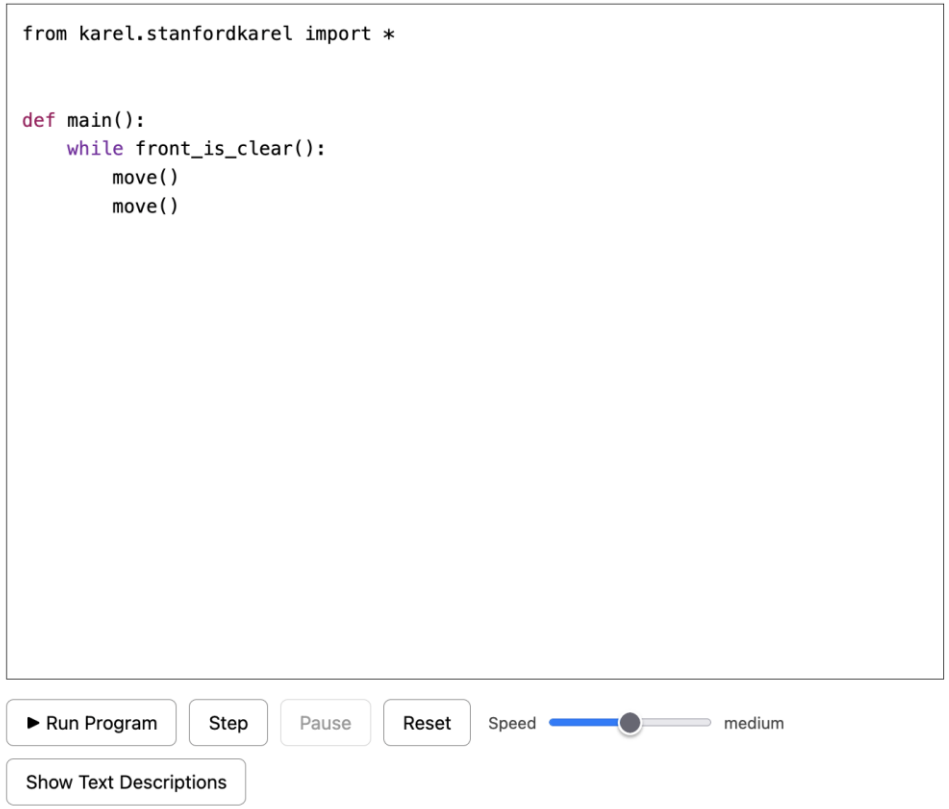
Line 5: Karel tried to pick up a beeper at (1, 1) but there isn't one there.

while loops only test *at the top of the loop*

- Check out this program. Does it crash?

```
from karel.stanfordkarel import *


def main():
    while front_is_clear():
        move()
        move()
```



while loops only test *at the top of the loop*

- Check out this program. Does it crash when run on a 4x6 world?

```
def main():
    while front_is_clear():
        move()
        move()
```

while loops only test *at the top of the loop*

- Check out this program. Does it crash?

```
def main():
    while front_is_clear():
        move()
        move()
```

YES! Why?
because Karel only
checks the front_is_clear()
condition **AT THE TOP OF THE LOOP**

Once it enters the loop, it doesn't check again until all statements in the loop have executed

defining new functions

- Karel has one big limitation – **they can't turn right**
- You can fix that limitation by **defining your own function**
- You do this using the `def` keyword

defining new functions

- How do you turn right when you only know how to turn left?

defining new functions

- How do you turn right when you only know how to turn left?

- turn left three times!

```
turn_left()
turn_left()
turn_left()
```

defining new functions

```
def turn_right():
    turn_left()
    turn_left()
    turn_left()

def main():
    turn_right()
```



use the def keyword to define a new function

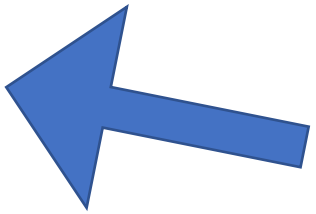


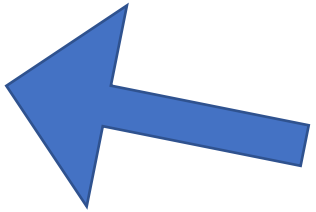
once you define your new function, you can use it just like the built-in functions

if statements new functions

```
def safe_move():
    if front_is_clear():
        move()

def main():
    safe_move()
```





An if statement is a lot like a while loop

However, it only runs *at most once*, and only if the condition is true

unlike just plain move(), safe_move() will never result in a crash

exercises

- Exercise 3: Write a program that successfully picks up all the beepers on *both* the "beeper pile" and "beeper piles" boards
  - (your program has to work if I run it on either of them)

- Exercise 4: Write a program that successfully removes *all* beepers from *any* board I run it on. I may test it on boards you don't have access to!

- When completed, put all exercises in one text file and mark what code goes with which exercise using comments

- **MAKE SURE YOU PRESERVE YOUR INDENTATION!!!!**